

Project 2

Assessing Potential Vulnerabilities by Analyzing Source Code of Networking Systems

Objective

To analyze an existing code base for potential vulnerabilities.

Description

- Students may work individually or form a team of up to four members to complete the project. While teamwork is allowed, students are encouraged to complete the project on their own.
- All students must submit their own answers to this project.
- **Both undergraduate and graduate students are required to answer all questions.**

Two Networking Systems - A DNS Relay and An Embedded System

1. The DNS Relay

Included with this project is a copy of the source code of a Java project called DenisDNS. DenisDNS is an open-source DNS relay that while capable of basic functionality has crucial security vulnerabilities. You will need to download the source from [CEG6430/DenisDNS at main · jzhang369/CEG6430 \(github.com\)](https://github.com/jzhang369/CEG6430)

This is a relatively small codebase so it should be highly accessible to you to perform code auditing. Nevertheless, if you want to expedite your analysis, you might want to prioritize your analysis for the following classes/methods.

- DnsServer.java: run(): The main execution loop of the program.
- DnsResolver.java: addressForHost(): The method where DennisDNS queries another DNS server when it cannot find the IP address it is trying to resolve in its own cache.
- DnsResponse.java: The class in which a response from the DNS server queried is parsed by the program for use, starting with the constructor.

For this project, you will **NOT** need to run DenisDNS. However, if you want to run this relay, observe its run time behaviors, the following information might be of help.

- A Java Runtime at version 7 or higher is required to run DenisDNS.
- A pre-compiled copy of DenisDNS is included as "DenisDNS.jar". Run it from the command line with "java -jar DenisDNS.jar".
- In order to test the server, the dig unix command (nslookup is an equivalent tool that can be used on Windows) can be used on the same computer with the command "dig @127.0.0.1 -p port domain" where port is the port number listed by DenisDNS when it is executed and domain is the domain name you wish to look up.

2. The Embedded System

Included with this project is a copy of the source code of a real-time operating system, which is modified from FreeRTOS, an open-source market-leading real-time operating system (or RTOS). We deliberately add some security flaws into the source code. You need to analyze this program and answer questions relevant to vulnerabilities. You can access the source code from [CEG6430/EmbeddedSystem/160919 FreeRTOS Labs - Broken at main · jzhang369/CEG6430 \(github.com\)](https://github.com/jzhang369/CEG6430-EmbeddedSystem-160919-FreeRTOS-Labs-Broken-at-main)

Since the codebase of this system is relatively large, we offer a list of files that are relevant to this project, hoping to expedite your analysis if needed.

- 160919_FreeRTOS_Labs-Broken/FreeRTOS-Plus/Source/FreeRTOS-Plus-TCP/FreeRTOS_TCP_WIN.c : This file implements the TCP window and congestion control.
- 160919_FreeRTOS_Labs-Broken/FreeRTOS-Plus/Source/FreeRTOS-Plus-TCP/FreeRTOS_TCP_IP.c: This file implements the TCP-IP protocol.
- 160919_FreeRTOS_Labs-Broken/FreeRTOS-Plus/Source/FreeRTOS-Plus-TCP/FreeRTOS_Sockets.c : Implement how the socket is initialized.
- 160919_FreeRTOS_Labs-Broken/FreeRTOS-Plus/Demo/FreeRTOS_Plus_TCP_Minimal_Windows_Simulator/main.c : The main file of FreeRTOS windows simulator.
- 160919_FreeRTOS_Labs-Broken/FreeRTOS-Plus/Demo/FreeRTOS_Plus_TCP_Minimal_Windows_Simulator/FreeRTOSConfig.h : Configuration file of FreeRTOS windows simulator application.

Your Name: Bhuwan Pandey

UID: U01155118

Q1. The DeniSDNS relay properly verifies whether the source IP address of the received DNS response is the same as the IP address of the DNS server to which the DNS query is sent.

True/False?

Answer: False

```
public HostRecord addressForHost(String domainName) throws Exception {  
    // NOTE: this function may block!  
    Random rand = new Random();  
    byte buffer[] = DnsRequest.constructPacket(rand.nextInt(), 0x100, domainName);  
    DatagramPacket p = new DatagramPacket(buffer, buffer.length);  
    p.setAddress(dnsServerAddr);  
    p.setPort(serverPort);  
    socket.send(p);  
    // receive  
    byte recvBuffer[] = new byte[1024];  
    p = new DatagramPacket(recvBuffer, recvBuffer.length);  
    socket.receive(p);  
    DnsResponse response = new DnsResponse(recvBuffer, p.getLength());  
}
```

The DnsResolver sends a query to the configured DNS server (dnsServerAddr) but when receiving the response via socket.receive(p), there is no check that the source IP address of the received packet matches dnsServerAddr. The DatagramPacket after receive contains the source address (p.getAddress()), but the code never compares it to the expected server address.

Q2. The DeniSDNS relay properly verifies whether the transaction ID of the received DNS response is the same as the transaction ID of the DNS query.

True/False?

Answer: False.

```
public HostRecord addressForHost(String domainName) throws Exception {  
    // NOTE: this function may block!  
    Random rand = new Random();  
    byte buffer[] = DnsRequest.constructPacket(rand.nextInt(), 0x100, domainName);  
}
```

The code generates a random transaction ID for the query using rand.nextInt() (which is cast to int but written as a short in the packet). When the response is received, it parses the transaction

ID from the response via `DnsResponse.getTxnId()`, but never compares it to the original query's transaction ID. The relay accepts any response regardless of transaction ID matching.

Q3. The DenisDNS relay properly verifies whether the domain name contained in the received DNS response is the same as the domain name of the DNS query.

True/False?

Answer: True.

```
public void parseQuestions(DataInputStream dis) throws Exception {
    for (int i = 0; i < numQuestions; i++) {
        DnsQuestion question = new DnsQuestion(dis);
        questions.add(question);
    }
}
```

The DNS response parsing extracts the domain name from the question section of the response. The `DnsResolver` then processes answers only for that domain name. However, note that the verification is implicit (the response must contain a question section with the domain name), not an explicit comparison against the original query domain. The code accepts what is in the response's question section rather than verifying it matches what was asked.

```
public class DnsQuestion {
    public DnsQuestion(DataInputStream dis) throws Exception {
        // read question name
        byte firstCount = dis.readByte();
        domainName = Utils.readDomainName(dis, firstCount);
    }
}
```

Q4. The DenisDNS relay will cache a DNS response

☐ for 360 seconds or 6 minutes.

☐ for 60 seconds or 1 minute.

[] for 180 seconds or 3 minutes.

[] indefinitely (i.e. forever) as long as the DenisDNS process is active.

Answer: for 360 seconds or 6 minutes.

When constructing a DNS response packet (for cached entries), the code sets the TTL to 360 seconds. This is the value that will be sent to clients, indicating how long they should cache the response. However, the relay's own internal cache does not implement TTL expiration, instead it caches indefinitely.

```
// answers
for (IPAddress address: ipAddresses) {
    dos.writeByte(v: 0xC0); // the domain name is a pointer
    dos.writeByte(v: 0x0C); // the pointer location
    dos.writeShort(Utils.QUERY_TYPE_A); // query type
    dos.writeShort(v: 1); // query class
    dos.writeInt(v: 360); // TTL = 6 minutes
    dos.writeShort(v: 4); // RR data length
    dos.write(address.getBuffer(), off: 0, len: 4); // IP address
}
```

Q5. The DenisDNS relay is vulnerable to DNS cache positioning attacks where a forged DNS response can be received, accepted, processed, and cached by this DenisDNS relay.

True/False

Answer: True.

Because the relay does not verify the source IP address nor the transaction ID, it is vulnerable to DNS cache poisoning.

```
socket.receive(p);
DnsResponse response = new DnsResponse(recvBuffer, p.getLength());
```

An attacker can send a forged DNS response that matches the domain name which is parsed from the response itself (not verified), will be accepted since no source IP or transaction ID validation occurs, will be cached by HostCache.java (which adds entries without checking TTL or authenticity), and last but not least will be served to subsequent clients until the relay process ends (since the cache has no expiration).

```
synchronized(hostCache) {
    for (DnsQuestion question: questions) {
        String host = question.getDomainName();
        HostRecord record = resolve(host);
        // send back response, record may be null.
    }
}
```

Q6. The random numbers generated by this TCP/IP stack implementation are predictable.

True/False

Answer: True.

The generator is deterministic, uses a predictable mathematical pattern, lacks cryptographic security, and is not properly seeded with unpredictable entropy, making its output sequence predictable to an attacker who observes prior values.

```
/*
 * Each time a new TCP connection is being made, a new Initial Sequence Number shall be used.
 * The variable 'ulNextInitialSequenceNumber' will be incremented with a recommended value
 * of 0x102.
 */
#define INITIAL_SEQUENCE_NUMBER_INCREMENT    ( 0x10L )
```

Q7. The initial value of the sequence number for a new TCP connection, when it is generated by this embedded system, is predictable.

True/False**Answer: True.**

Since the underlying random number generator is predictable (for the same reason as above), the TCP ISNs derived from it are also predictable, enabling attacks such as TCP session hijacking or spoofing.